Loup M. Marquez Luttringer – Project Manager

Matheus Lopez – Back-End & Front-End Developer

Noah Gibelli – AI Developer

Version: v1.0

Date: October 22, 2025

Horus Project – Orchestrator Implementation Justification

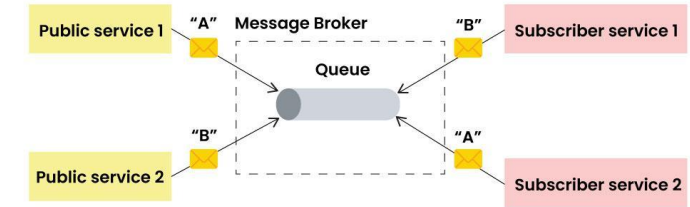
(Valorizing Architecture, Adopting the best technologies for our architecture)

Point 1 – A – Valorize Architecture

When we started to design our Horus project, one of the biggest points was to create a tool that can manage multiple AI agents safely, efficiently and as fast as possible. This idea defines in fact the concept of orchestration. Orchestration is a way to interconnect every element, like User interface or agents, to a central brain.

At this point, we know that if we want a large number of people to request this central brain, we have to take care of the architecture of the code. So, at first time, we had to select the best architecture as possible for our project. We have tested several types of architecture. The first architecture was monolithic architecture. This one consists of developing a big block of code. It corresponds to a code built as a unified unit. But, with this type of architecture, we have seen that you can directly have a big variety of problems when you want to put it in production because a unified code needs to be updated entirely at each new features or updates. The fact is that a lot of clients will wait for an answer so if each client makes a request at the same time to an LLM, it will be time and resource consuming, and the program could go down. Running AI functions in one big progress was not the most optimized architecture for our process. Our need was to have a central brain that can manage agents that work independently. For that reason, we have oriented our research to microservices and message driven architectures. After testing different prototypes, we have seen that a distributed message-based system was the best solution to deal with containerize agents. This solution gives us the opportunity to obtain a real modular and isolated infrastructure. The orchestrator delegate tasks, managing client's requests and assures a link between client and agents without computing anything.

This system consists of exchanging by message and not direct calls. Messages are sent from a producer to a message broker, which then routes them to the intended consumer.



Message Queue Point to Point

Instead of blocking a part of your process because a client waits for an answer. In this case, a client will ask something and when the agent finishes producing the answer, finally the client will have an answer. The client front-end will only wait for an answer. A central containerized component will receive messages from clients and will distribute them to agents. In this case, each agent will read requests from clients and work independently inside a single container. Also, we can have a normalized input for the message, so this let us a big flexibility. And finally, when a message crashes, it will not impact on the other subprocess and block the chain.

We also analyzed other orchestration frameworks such as Nvidia Agent toolkit. While they are powerful, most of them are designed for cloud workflows or pipeline automation rather than real-time multi-agent communication.

Our API combined with a message broker gave us exactly what we needed: real-time communication, asynchronous execution, and simple deployment in Docker containers. It's a more flexible architecture where each part can be customized without wasting too much time.

Point 2 – B – Adopting the best technologies for our architecture

To have a robust solution that can put in place our choice of architecture. Choosing the best technologies that can shape our architecture was an important part of the project. First of all, in order to build our orchestrator as a service, we choose to create an API in Python. For this part, we have chosen FastAPI solution that gives us an opportunity to build a fast, asynchronous REST and Websocket backend. The goal here is to use modern libraries that contain modern syntax, automatic documentation and great compatibility with our AI Stack. The main objective is to save time at each floor of this project.

For the message broker, we have chosen RabbitMQ that provides reliable message delivery, retry mechanisms, and scalability. The solution is containerized so it makes the whole system fault-tolerant: for example, if an agent crashes, haven't read correctly a message we can use a retry method.

Concerning the persistence of the data and for caching, we have chosen to use Redis and PostgreSQL. The role of Redis will stock a unique id, provided at each login, when a user wants to logout. In this case, we will be sure that a disconnected user will never have the opportunity to use the Api until he wants to log in again. The role of PostgreSQL will handle long-term data (users' information, logs, messages). But why these two databases? Concerning Redis, it's because each entry inside has a time living of one hour that is perfect concerning storage saving and it's the best solution to shape a cache. Concerning PostgreSQL, we choose to use this one instead of MongoDB because this solution can allow us to request structured database. Instead of taking the time to decorticate data collect at each request, we can target specific data.

Concerning interactions between the client and the orchestrator we have chosen to use WebSocket. It can allow us to have real-time communication between the front-end dashboard and the orchestrator. It allows users to see agent responses live, as if they were chatting directly with the AI. It's faster than a simple HTTP call and in this way, it's opportunity to remember how to contact each client. Each client will follow his proper conversations in real time, no need to refresh.

For security, we implemented authentication using JWT tokens, where each user has a defined role. Only admins can manage the system, while normal users can access conversation and analysis features. This ensures that even if the system is self-hosted, every access remains controlled and auditable. And it’s also a secure way to access users’ information inside the API. Finally, the API can be secured and adapted to each user.